

Exercícios: 4 Pilares da Programação Orientada a Objetos em Python

1. Criação de Classes e Objetos (Abstração)

Crie uma classe `Carro` com os atributos `marca`, `modelo` e `ano`. Adicione um método `exibir_dados()` que imprime as informações do carro. Em seguida, crie um objeto dessa classe e chame o método.

```
# Exemplo esperado de uso:
carro1 = Carro("Toyota", "Corolla", 2020)
carro1.exibir_dados()
```

2. Encapsulamento com Getters e Setters

Crie uma classe `ContaBancaria` com os atributos privados `__saldo` e `__titular`. Adicione métodos getters e setters para acessar e modificar esses atributos, garantindo que o saldo nunca seja negativo.

```
# Exemplo esperado de uso:
conta = ContaBancaria("João", 1000)
conta.depositar(500)
conta.sacar(200)
print(conta.get_saldo()) # Deve exibir 1300
```

3. Herança

Crie uma classe `Pessoa` com os atributos `nome` e `idade`, e um método `cumprimentar()` que exibe uma mensagem de saudação. Depois, crie uma classe `Estudante` que herda de `Pessoa` e adiciona o atributo `matricula`. Modifique o método `cumprimentar()` para incluir a matrícula na saudação.

```
# Exemplo esperado de uso:
estudante = Estudante("Ana", 21, "20230001")
estudante.cumprimentar()
```

4. Polimorfismo com Métodos

Crie uma classe `FormaGeometrica` com um método `calcular_area()`. Crie as subclasses `Retangulo` e `Circulo`, implementando o cálculo da área para cada forma.

```
# Exemplo esperado de uso:
retangulo = Retangulo(5, 10)
circulo = Circulo(7)
```

```
print(retangulo.calcular_area()) # Exibe a área do retângulo
print(circulo.calcular_area())  # Exibe a área do círculo
```

5. Sobrecarga de Construtores (Simulação)

Em Python, simule a sobrecarga de construtores criando uma classe **Produto**. Permita que ela possa ser inicializada apenas com o nome do produto, ou com o nome e o preço.

```
# Exemplo esperado de uso:
produto1 = Produto("Notebook")
produto2 = Produto("Celular", 2500)
```

6. Composição de Classes

Crie uma classe **Endereco** com os atributos **rua**, **cidade** e **estado**. Crie uma classe **Pessoa** que contém um objeto do tipo **Endereco** como atributo.

```
# Exemplo esperado de uso:
endereco = Endereco("Rua A", "São Paulo", "SP")
pessoa = Pessoa("João", 30, endereco)
pessoa.exibir_dados()
```

7. Método Estático

Adicione um método estático à classe **Calculadora** que calcula o valor de um número elevado ao quadrado.

```
# Exemplo esperado de uso:
print(Calculadora.elevar_quadrado(4)) # Deve exibir 16
```

8. Classe Abstrata

Usando o módulo **abc**, crie uma classe abstrata **Funcionario** com um método abstrato **calcular_salario()**. Depois, implemente subclasses **Gerente** e **Operador** que definam o cálculo do salário de forma específica.

```
# Exemplo esperado de uso:
gerente = Gerente("Maria", 5000)
operador = Operador("José", 20, 50)
print(gerente.calcular_salario())
print(operador.calcular_salario())
```

9. Uso do **super()** em Herança

Crie uma classe `Animal` com o método `fazer_som()` que exibe "Som genérico". Crie uma subclasse `Cachorro` que sobrescreve esse método, mas também chama o método da superclasse.

```
# Exemplo esperado de uso:  
cachorro = Cachorro()  
cachorro.fazer_som()
```

10. Sobrecarga de Operadores

Crie uma classe `Ponto` que representa coordenadas 2D e sobrescreva o operador `+` para que seja possível somar dois pontos.

```
# Exemplo esperado de uso:  
ponto1 = Ponto(1, 2)  
ponto2 = Ponto(3, 4)  
resultado = ponto1 + ponto2  
print(resultado) # Deve exibir algo como "(4, 6)"
```